SPECULATE: Generating REST API Specifications Using LLMs

KRISHANU SINGH , Indian Institute of Technology, Delhi, India
KUSHAGRA KARAR , Indian Institute of Technology, Delhi, India
ABHILASH JINDAL , Indian Institute of Technology, Delhi, India
GUOWEI YANG , University of Queensland, Australia

REST APIs are widely used for accessing web services. To support their use and testing, developers often write API specifications in open formats like OpenAPI. However, writing these specifications manually is tedious and error-prone, leading to incomplete or outdated specifications that can hinder both API usage and automated testing. Existing tools attempt to generate API specifications from source code using static or dynamic analysis. Unfortunately, these tools are often tightly coupled to specific languages and frameworks, making them hard to generalize and extend.

This paper presents SPECULATE, the first approach to combine lightweight static analysis with large language models (LLMs) to automatically generate REST API specifications from source code. By leveraging LLMs trained on diverse codebases, SPECULATE generalizes easily across languages and frameworks. We evaluate SPECULATE on 19 real-world web repositories spanning two programming languages and three REST frameworks. Our results show that SPECULATE achieves competitive precision while significantly improving recall (by +46.6%) over existing tools.

1 INTRODUCTION

REST APIs form the backbone of modern web services. When paired with an OpenAPI Specification (OAS), they enable a powerful ecosystem of automated tools—from generating client libraries in dozens of languages [31] to building test suites that validate web services with minimal manual effort [13, 18]. However, prior work has shown that developer-provided specifications are often incomplete or outdated [6, 7, 13, 17].

To address this, researchers and practitioners have developed automated tools to generate API specifications directly from source code. Broadly, these tools fall into two categories:

(1) Dynamic analysis approaches [3, 25, 36]. These tools require developers to deploy the API and exercise its endpoints. They automatically capture request–response pairs and use them to construct specifications. While effective in some scenarios, this approach burdens developers with the task of invoking all endpoints with diverse inputs, often resulting in incomplete coverage. Moreover, these tools typically lack support for parameter constraints and response constraints.

(2) Static analysis approaches [5, 17, 27, 29, 30, 32, 38]. These tools examine source code to infer endpoints and their request–response structures. Although they can, in principle, discover all endpoints, they usually rely heavily on developer-provided annotations and framework conventions. Even small deviations from expected templates can lead to incorrect specifications.

Recent works [17, 19] advance the static analysis frontier by combining static and symbolic analysis for Java programs. It can generate accurate specifications even when developers diverge from framework templates. However, these works still produce inaccurate specifications due to (1) analysis boundaries, such as calls into native code through Java-native interface (JNI), (2) dynamic behaviors like dependency injection and reflection, and (3) path explosion caused by multiple conditional branches.

Authors' addresses: Krishanu Singh , Indian Institute of Technology, Delhi, New Delhi, India, krishanu.visitor@cse.iitd.ac.in; Kushagra Karar , Indian Institute of Technology, Delhi, New Delhi, India, csy247554@cse.iitd.ac.in; Abhilash Jindal , Indian Institute of Technology, Delhi, New Delhi, India, ajindal@cse.iitd.ac.in; Guowei Yang , University of Queensland, Brisbane, Australia, guowei.yang@uq.edu.au.

Static-analysis tools help developers avoid having to manually exercise every API endpoint, but they are tightly coupled to the language in which the code is written. Adding support for other languages and frameworks—such as Django for Python and Express for Node.js—requires substantial development effort. Even more challenging is extending these tools to capture richer specification details, such as constraints on the responses.

In this paper, we present Speculate, a new approach that combines lightweight static analysis with large language models (LLMs) to generate OpenAPI specifications from source code. Speculate first performs lightweight static analysis to build a *code index*. The index maps program symbols (e.g., functions, classes) to their definitions and properties (e.g., class hierarchies). Using the index, Speculate collects relevant code context for each OpenAPI component and endpoint, and prompts an LLM to generate the corresponding specification. Notably, only the code indexing depends on the programming language and framework; the latter two are language-agnostic.

We evaluate Speculate on 19 real-world repositories spanning two programming languages (Java and Python) and three web frameworks (Jersey, Spring Boot, and Django). We use DRF-Spectacular [14] as a baseline for Django repositories and Respector [17] for Jersey and Spring Boot repositories. We address the following research questions:

RQ1: Can Speculate generate accurate specifications? For the Django repositories, Speculate has much better precision and recall than DRF-spectacular across all the dimensions. For Jersey and Spring Boot repositories, Speculate usually has slightly worse precision but better recall than Respector for all the repositories across all the dimensions. For responses, Speculate has significantly better recall. On average, the difference in precision and recall for Django, shown as ($p\%, r\%$), are as follows: request parameters (+43%, +38%), request constraints (+53%, +49%), response parameters (+21%, +29%), and response constraints (+24%, +24%). For Java based frameworks, the difference in precision and recall, shown as ($p\%, r\%$), are as follows: request parameters (+12%, +45%), request constraints (+9%, +47%), response parameters (+33%, +71%), and response constraints (+24%, +70%). We further diagnose reasons for why Speculate generates inaccurate specifications.

RQ2: How does the choice of LLM—reasoning vs. non-reasoning—affect Speculate's accuracy and usage cost? We find that reasoning models provide only slightly higher accuracy than non-reasoning models, but with significantly higher cost. For example, GPT-o1 costs ~$100 per repository versus ~$1 for GPT-4.1 Mini, while the average execution time per repository ranges from 13 minutes (GPT-4.1 Mini) to 80 minutes (Deepseek-R1). We examine the rare cases where reasoning models provide better accuracy, and find that this happens when the code diverges significantly from the framework template.

Our contributions are as follows:

- We design and implement Speculate, a modular system that combines lightweight static analysis with LLMs, and release it as open-source software [1].
- We demonstrate that Speculate can be easily adapted to multiple programming languages and web frameworks by applying it to 19 real-world repositories written in two programming languages (Python and Java) and three web frameworks (Django, Jersey, and Spring Boot).
- We compare Speculate with state-of-the-art static analysis tools on the 19 repositories. Our evaluation shows that Speculate achieves competitive precision while significantly improving recall (+46.6%), compared to state-of-the-art static and symbolic analysis tools.

2 MOTIVATION

Automatically generating accurate specifications for REST APIs is challenging. Many techniques [17] have been recently proposed but they often fall short in practice due to limitations in coverage, scalability, or adaptability to different languages or frameworks. To motivate the need for our

approach, we present three representative examples from popular web applications that illustrate the key shortcomings of state-of-the-art tools. In particular, we highlight inconsistencies or omissions introduced in the specifications generated by (1) DRF-Spectacular [14], a widely adopted lightweight static analysis based tool for Django repositories and (2) Respector [17], a more sophisticated static analysis based tool for Jersey and Spring Boot repositories. We begin by providing essential background before discussing these examples.

2.1 Background

Modern web applications use REST frameworks, like Django, Jersey and Spring Boot, to expose APIs over the web to enable standard CRUD (Create, Read, Update, Delete) operations. Each of these operations is exposed as an *endpoint*: a URI path with an HTTP method (e.g., GET and POST). Endpoints accept a request *parameters* and produce a *response*. Parameters are of two broad types: (1) *query and path parameters* are typically small and are passed in the URI, and (2) *body parameters* are typically large JSON objects passed in the body of the request. If the parameters are valid, the endpoint performs the requested operation, and generates a successful response with a HTTP status code of 2XX. Other HTTP status codes indicate error (*e.g.,* 4XX indicates a malformed request).

An OpenAPI specification is a YAML file containing a description of each endpoint, including a description of its parameters and responses, along with status codes. Both parameters and responses have a *schema* that describes the *type* of the object. Types can be primitive, such as `string`, or refer to custom *components* which represent complex JSON structures.

Since endpoints validate requests, the specification includes *parameter constraints*, e.g., a parameter is required and must have a minimum value of 3. Similarly, the specification describes *response constraints* to validate responses. Automated tools can use OpenAPI specifications to generate client code in various programming languages [31] and generate automated tests to verify the implementation [13, 18].

For example, Listing 1 shows the implementation of a GET API `/api/failures` from Mozilla's Django-based TreeHerder repository [21]. TreeHerder manages continuous integration runs of various Mozilla projects, including Firefox. TreeHerder registers the API with Django at line 4, specifying that it is implemented by `class Failures`. To implement the API, `class Failures` overrides Django's `ListAPIView` class at line 9. If the query parameters are invalid, the API returns an error response with HTTP status code 400 (lines 15 to 17). Otherwise, it returns a successful response based on query parameters at line 30.

The corresponding OpenAPI specification for this API is shown in Listing 2. It specifies that `startday` is a query parameter with a parameter constraint: `required: true`. It specifies two possible responses: one with status code 200 and another with 400. The 200 response has a component.

2.2 Example 1: Deviations from template

While the API in Listing 1 expects several query parameters like `startday`, `endday`, and `tree` (project name) as shown in lines 33 to 35, the specification generated by DRF-spectacular omits these query parameters. This omission occurs because DRF-spectacular only inspects declarative properties of the view, such as `serializer_class = FailuresSerializer` at line 11. However, the API implementation instantiates a different serializer `FailuresQueryParamsSerializer` at line 14 and then uses it to validate and accept query parameters, as shown at line 15.

Our study of various public repositories (discussed in Section 4) reveals that developers often deviate from the framework suggested templates (like using a different serializer than the declared `serializer_class` in the example). To handle such deviations, researchers have developed more advanced static analysis tools such as Respector [17]. Respector tracks uses and defines variables of interest (such as `request.query_params` at line 14), tracks aliasing, *etc.* on the program paths.

```python
# --- File: treeherder/webapp/api/urls.py
from django.urls import include, re_path
urlpatterns = [
  re_path(r"^failures/$", intermittents_view.Failures.as_view()),
  # ...
]

# --- File: treeherder/webapp/api/intermittents_view.py
class Failures(generics.ListAPIView):
  """List of intermittent failures by date range and repo"""
  serializer_class = FailuresSerializer

  def list(self, request):
    query_params = FailuresQueryParamsSerializer(data=request.query_params)
    if not query_params.is_valid():
      return Response(data=query_params.errors,
                      status=HTTP_400_BAD_REQUEST)

    startday = query_params.validated_data["startday"]
    endday, repo = # ...
    self.queryset = (
      BugJobMap.failures.by_date(startday, endday)
      .by_repo(repo)
      .values("bug__bugzilla_id")
      .annotate(bug_count=Count("job_id"))
      .values("bug__bugzilla_id", "bug_count")
      .order_by("-bug_count")
    )
    serializer = self.get_serializer(self.queryset, many=True)
    return Response(data=serializer.data)

class FailuresQueryParamsSerializer(serializers.Serializer):
  startday = serializers.DateTimeField(format="%Y-%m-%d")
  endday = serializers.DateTimeField(format="%Y-%m-%d")
  tree = serializers.CharField()
  # ...
```

Listing 1. Implementation of the GET /api/failures/ endpoint in Treeherder.

```yaml
/api/failures/:
  get:
    parameters:
      - name: startday
        in: query
        required: true
        schema:
          type: string
          format: date-time
      # ...
    responses:
      '200':
        $ref: '#/components/responses/ListOfFailuresResponse'

components:
  responses:
    ListOfFailuresResponse:
      description: Successful response with a list of failures
      content:
        application/json:
          schema:
            type: array
            items:
              type: object
              properties:
                bug_count:
                  description: The count of jobs associated with the bug
                  type: integer
                bug_id:
                  description: The ID of the associated bug
                  type: integer
                # ... other properties for a failure object
```

Listing 2. OpenAPI specification for GET /api/failures/ endpoint, using a referenced component for the 200 response.

However, this deep static analysis gets tightly coupled toto the specific programming language. For example, Respector is implemented in Soot for Java, and adapting it to Python-based Django framework is non-trivial. Further, as we will show later, Respector has standard limitations of static analysis.

2.3 Example 2: Dynamic control flow and knowledge boundaries

Listing 3 shows the implementation of `GET /users` API from a popular road traffic analytics platform enviroCar [33], implemented using Jersey in Java. While the API takes pagination query parameters like `page` and `limit`, the Respector-generated specification fails to include them.

This omission occurs because the developer does not use standard Jersey annotations *e.g.*, `@QueryParam` to get pagination-related query parameters. Instead, `class UsersResource` calls the method `getPagination()` at line 6. This method is defined in its parent class `AbstractResource` which delegates the call to `this.pagination.get()` at line 23. The `pagination` field is of type `PaginationProvider` (line 13), an interface. This field is injected by Guice [16], a popular dependency

```java
// --- File: .../server/rest/resources/UsersResource.java
public class UsersResource extends AbstractResource {
  @GET
  public Users get() throws BadRequestException {
    // 1. Endpoint calls a method inherited from its parent class.
    return getUserService().getUsers(getPagination());
  }
}

// --- File: .../server/rest/resources/AbstractResource.java
public abstract class AbstractResource {
  // 3. The field is a reference to an Interface.
  private PaginationProvider pagination;

  // 4. Guice @Injects an implementation at runtime.
  @Inject
  public void setPagination(PaginationProvider pagination) {
    this.pagination = pagination;
  }

  // 2. The method simply delegates to the field.
  protected Pagination getPagination() throws BadRequestException {
    return this.pagination.get();
  }
}

// --- File: .../server/rest/pagination/PaginationProviderImpl.java
public class PaginationProviderImpl implements PaginationProvider {
  private final UriInfo uriInfo;

  // 6. Guice @Injects uriInfo
  @Inject
  public PaginationProviderImpl(UriInfo uriInfo, ...) {
    this.uriInfo = uriInfo;
    // ...
  }

  // 5. Extract pagination query parameters from uriInfo
  @Override
  public Pagination get() throws BadRequestException {
    return or(getPaginationFromRangeHeader(), getPaginationFromQueryParam()).orElse(getDefaultPagination());
  }
}
```

Listing 3. Key files showing how pagination parameters are handled via dependency injection in the enviroCar API.

injection framework, as shown at line 16. Guice injects an object of `class PaginationProviderImpl`. The actual query parameter handling logic resides in this class as shown at line 41, where the class gets pagination query parameters from another field `uriInfo`. This field is of type `javax.ws.rs.core.UriInfo`, which is also injected by Guice, as shown at line 32.

Mechanisms such as dependency injections and reflection obscure object-class relationships, making them difficult to resolve for static analysis. This is why Respector misses pagination related query parameters from this API. More generally, static analysis based tools need to explicitly build support for patching complicated control flows like dependency injections and reflections. Maintaining such support is tedious especially due to code evolution and third-party libraries. For instance, in this example, the developer chose to use a different third-party dependency injection framework, Guice, with subtle differences from the Jersey-provided dependency injection framework HK2 [12]. Building support in static analysis based tools for each such deviation from the standard template, and keeping this support up-to-date is tedious.

Moreover, Respector is limited to analyzing code within the target repository. However, key classes such `javax.ws.rs.core.UriInfo` is not part of the repository's code in this example. Not

analyzing useful library code, which may be written in a different language leads to knowledge boundaries, and thereby loss of specification accuracy, in static analysis based tools.

2.4 Example 3: Path explosion

Deep static analysis tools need to carefully handle path explosions to generate specifications from real-world repositories within a practical runtime budget. For example, Respector prunes paths using various strategies: (1) it integrates a Z3 SMT solver [10] to identify and prune non-feasible branches, (2) it ignores all back-edges and unrolls loops just once, and (3) it drops paths with recursive calls. Even with these strategies, the number of remaining paths may still be intractably large. Therefore, Respector limits the number of explored paths to 5,000, but such path pruning can introduce inaccuracies in the generated specifications.

For instance, in the `GET /entity-network` endpoint in the Senzing repository [26] (code is not shown due to space limit), Respector incorrectly generates a parameter constraint `required: true` for the query parameter `entities`. This mistake occurs because Respector pruned paths where the field `entities` was not required.

Managing and integrating SMT solvers like Z3 with core tool logic is non-trivial. Further, SMT solvers can bring challenges due to their limited vocabulary, *e.g.,* Z3 treats strings as scalars and cannot represent string query parameter splitting operations. In contrast, building Speculate did not require any deep expertise in static analysis and SMT solvers, making it more practical to maintain by citizen developers. To illustrate this point, we added support for response constraints to Speculate by adding new instructions like `"f. Determine required fields for (simple_name): - Check for validation annotations like @NotNull, @NotBlank, @NotEmpty, @Size(min=1)"` to the prompts. Respector, however, lacks this support, and adding such support is be non-trivial.

2.5 Summary

Industry tools (*e.g.,* DRF-spectacular for Django, tsoa for Typescript) use lightweight static analysis to automatically generate specifications from code. While easy to maintain, these tools lose accuracy whenever developers deviate from standard templates provided by the REST frameworks (Example 1). This has motivated researchers to perform deep static analysis, as in Respector [17].

However, building deep static analysis tools require expertise (such as integrating Z3 in Respector), making them difficult to maintain and extend (*e.g.,* it is quite difficult to add response constraints in Respector). Moreover, they suffer from the fundamental limitations of static analysis like handling dynamic control flows and external dependencies (Example 2) and managing path explosions (Example 3). Adapting deep analysis tools to new frameworks and languages is also challenging.

In contrast, this paper demonstrates that by using just lightweight static analysis along with LLMs, we can build a tool that easily works across programming languages and frameworks. LLMs are already trained on large codebases across diverse languages, frameworks, and libraries. Given the relevant code context, our proposed tool Speculate can correctly generate API specification by calling LLMs, without requiring any specific support for third-party libraries and dependency injection. For complex programs, there is no need to prune paths as done in deep static analysis; instead, it just needs to control the size of each prompt to fit into the context window of an LLM.

3 APPROACH

In this section, we present our approach, Speculate, which uses a lightweight static analysis and then leverages LLMs for generating specifications.

Figure 1 shows the overview of Speculate. Speculate works in three phases:

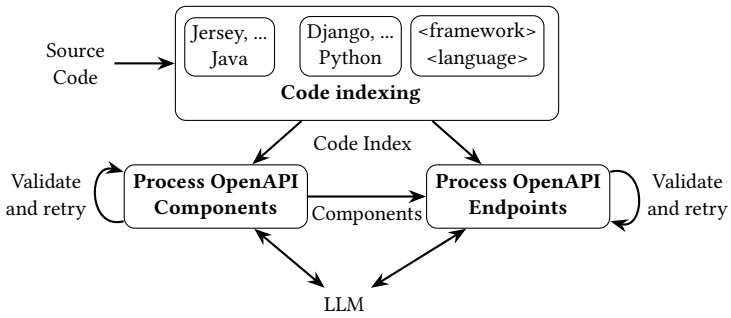


Fig. 1. SPECULATE works in three phases

(1) **Code Indexing**: SPECULATE analyzes the codebase using lightweight static analysis to generate a code index. This code index can be later queried using program symbols, such as functions and classes, to locate their definitions and to find other symbol properties, like class hierarchy.
(2) **Process OpenAPI Components**: SPECULATE uses the code index to collect the code context of each component and uses LLMs to build the component section of the specification.
(3) **Process OpenAPI Endpoints**: SPECULATE processes endpoints using the same method applied to components, with additional validation to ensure the integrity of the specification by checking for invalid component references.

3.1 Code Indexing

In this phase, SPECULATE performs lightweight static analysis to build a code index. The code index can be later queried using symbol names, such as functions, classes, methods, fields, variables, and annotations/decorators, for finding symbol definitions and other symbol properties like class hierarchy, classes in files, methods in classes, annotations/decorators over classes, access modifiers of fields, and callers/callees of functions and methods.

Code indexing is performed by language-specific code analyzers. These language-specific code analyzers are designed to be reusable when extending SPECULATE to other frameworks within the same language ecosystem. We have implemented code analyzers for two languages: a Python analyzer using the ast library and a Java analyzer using Soot [34] and JavaParser [15].

Code index, generated by a language-specific code analyzer, is further enhanced by framework-specific code analyzers to enable listing and querying code definitions for OpenAPI components and endpoints. We have implemented three framework-specific code analyzers: Spring Boot analyser, Jersey analyzer and Django analyzer.

Our Jersey and Spring Boot analyzers identify endpoints by statically detecting REST annotations (e.g., `@Path`, `@RestController`) and their associated HTTP method mappings to determine the exact handler method for each URL. In contrast, since Django can register endpoints dynamically, the Django analyzer in SPECULATE modifies Django's EndpointEnumerator class and then starts the server to find all the registered endpoints, HTTP methods, and their associated entrypoints. For component detection, the Django analyzer marks `Model` and `Serializer` classes as components. Conversely, the Jersey and Spring Boot analyzers employ a top-down approach: starting from the discovered endpoint handlers, they identify all referenced Data Transfer Objects (DTOs) and recursively trace their full dependency graph—including parent classes and field types—to build a complete data model.

This is the only phase of the approach that is dependent on the specific programming language and framework.

3.2 Process OpenAPI Components

The component specification defines reusable schema objects, and thus is generated first to ensure consistency and reusability across the API documentation. In this phase, Speculate uses the code index to identify all the component classes. Next, Speculate constructs a dependency graph among components using the code index, to sort all the components in a topological order. Next, it sends the relevant code for each component, following this order, to the LLM to generate its OpenAPI specification. To handle name collisions, where distinct components share a simple name like User, Speculate uses each component's unique source path as a canonical identifier. The LLM is prompted to return this identifier as metadata, allowing Speculate to detect true collisions and generate an unambiguous, context-derived schema name.

For Django, the code index returns the following relevant code for a component class: the primary model class (identified using `Meta` inner class of serializers), any related model classes referred via `ForeignKey`, `ManyToManyField`, *etc.*, recursively discovered parent classes, and nested serializers.

For Jersey and Spring Boot, the analyzer constructs a component's context by recursively gathering its parent classes and the types of its fields. To accurately model polymorphism, it also resolves any referenced interfaces to their concrete implementing classes, ensuring the schema reflects the actual objects used at runtime.

All the LLM prompts in Speculate are designed with a structured format. Following shows the prompt template to generate specification for OpenAPI components:

```
1. Task Description
You are an expert in [framework]. Your task is to map Java types to OpenAPI schemas, inferring
    ↪ metadata like required/readOnly from source annotations.

2. Code Context
[Source code for the target class and its full inheritance hierarchy]

3. Available Schemas
[List of previously generated component schemas available for use with $ref]

4. Output Format Rules
- Each schema must contain type: object and a properties field.
- Use $ref to link to other component schemas.
...

5. Quality Notes
- Ensure strict conformance to OpenAPI 3.0.
...
```

3.3 Process OpenAPI Endpoints

After generating component specifications, Speculate proceeds to generate endpoint specifications in this phase. Similar to the previous phase, this phase sends relevant code for each endpoint to the LLM to generate its specification. While collecting code context, we gather relevant code with varying nesting depth, collecting code of data classes to more depth than utility classes.

In the code index, the Django analyzer adds code for serializers (using `serializer_class` assigned in the `ViewSet` or `APIView` class), adds "plugin" components like `pagination_class`, `filter_backends` and `permission_classes`, along with the code for the endpoint class and the code for referred classes and functions.

For Jersey and Spring Boot, the analyzer gathers relevant DTOs and their dependencies, then adds framework-specific context: Jersey's analysis includes custom serialization providers (e.g., MessageBodyWriter), while Spring's includes the injected service-layer dependencies (e.g., @Autowired services) responsible for creating the response data.

To decompose the task of generating endpoint specification, we prompt the LLM separately to generate request and response specifications. The LLM prompt for endpoint processing follows a structured format similar to the component prompt. For example, the following shows the prompt template for generating request specifications for Jersey/Spring Boot:

```
1. Task Description
You are an expert in [framework]. Your task is to generate the parameters and requestBody sections
    ↪ by analyzing annotations on a controller method.

2. Code Context
[Source code for the REST controller class, the target endpoint method, and any associated
    DTOs/POJOs]

3. Available Schemas
[List of previously generated component schemas available for use with $ref]

4. Framework Specific Generation Logic & Rules
- Identify parameter location (path, query, etc.) from method annotations (e.g., @PathParam).
- Identify the request body from the un-annotated POJO parameter in the method signature.
- The request body's schema must use a \$ref to an available component.
...

5. Quality Notes
- Ensure strict conformance to OpenAPI 3.0.
...
```

Whenever LLMs generate a new part of the specification, SPECULATE does three types of **validations**: (1) *structural integrity*: SPECULATE uses a YAML parser to check that the specification is a valid YAML and does not have empty properties, (2) *referential integrity*: SPECULATE checks that all the `$ref` pointers refer to known components,[1] (3) *OpenAPI integrity*: SPECULATE checks OpenAPI-specification rules, like `paths` must be a dictionary and `parameters` must be a list. Whenever possible, SPECULATE fixes invalid specifications, such as add a missing colon to make it a valid YAML. Otherwise, SPECULATE appends the caught error to the prompt and retries calling the LLM.

Overall, the implementation of SPECULATE consists of approximately 12k lines of Python code, structured into framework-specific modules and a reusable core. The reusable core is responsible for LLM integration and prompting, specification parsing and validation, and overall orchestration. Adapting SPECULATE to a new framework requires only the implementation of a framework-specific code indexer, as shown in Figure 1.

4 EVALUATION

We evaluate SPECULATE by investing the following two research questions:

RQ1: Can SPECULATE generate accurate specifications?
RQ2: How does the choice of reasoning and non-reasoning LLMs impact SPECULATE's accuracy?

4.1 Evaluation Setup

Dataset. We use 4 Django, 8 Jersey and 7 Spring Boot repositories for our evaluations, as shown in Table 1. All 8 Jersey and 7 Spring Boot repositories were also evaluated in the prior Respector [17] study. For Django, the selection criteria emphasized repositories with significant adoption (at least 200 GitHub stars) and active maintenance, judged by recent commits. As shown in Table 1, these

[1]Recall that we process components before endpoints and we process components in the topological order.

Table 1. Diversity in code size, age, and popularity among the 19 Django, Jersey, and Spring Boot repositories used for evaluation.

Repository	KSLoC (Spec)	Stars	Age (yrs)	Description	End pts
Django-Based					
Treeherder	30.8 (15.1)	262	16	CI data mgmt	83
Mathesar	34.9 (8.7)	4.2K	4	Postgres DB platform	105
Education backend	11.0 (1.9)	445	6	LMS	31
Libre Photos	23.5 (9.6)	7.4K	8	Auth service	159
Jersey-Based					
enviroCar	22.7 (5.9)	31	12	Road traffic analytics	128
Features Service	1.0 (0.4)	36	4	Feature models API	18
REST Countries	1.3 (0.9)	36	4	Country info API	27
Senzing	30.9 (3.3)	8	6	Entity resolution API	34
Digdag	54.8 (2.1)	1.3K	10	Workload automation	41
Gravitee.io	118.8 (2.1)	296	10	API mgmt tool	28
Jersey-Based (cont.)					
Kafka REST Proxy	19.4 (4.3)	122	10	Kafka cluster API	74
Cassandra	10.5 (1.7)	9.4K	16	NoSQL DB API	50
Spring-Boot-Based					
CatWatch	4.0 (0.7)	59	10	GitHub data API	14
CWA Verif. Server	2.1 (0.2)	1.9K	5	Corona-Warn-App API	5
OCVN	24.6 (39.9)	5	10	Vietnam public procurement API	278
Ohsome	7.3 (20.4)	54	7	OpenStreetMap API	159
ProxyPrint	5.5 (2.1)	6	9	Printshop API	75
Quartz Manager	2.3 (0.3)	261	9	Quartz Scheduler API	10
Ur-Codebin	1.2 (0.4)	2	4	Code sharing API	7

repositories range from 1k to 118.8k lines of code, with 0.3k to 39.9k lines of specifications for 5 to 278 endpoints. They have been in development for 4-16 years and serve diverse domains like education, healthcare, logistics, and software engineering.

Baselines. We compare our results with established tools for each framework. For Django repositories, we use DRF-spectacular, a lightweight static analysis tool, officially recommended by Django [14]. For Jersey and Spring Boot, we compare against Respector [17], a state-of-the-art tool based on sophisticated static analysis.

Ground Truth. The Jersey and Spring Boot repositories have developer-provided specifications. However, developer-provided specifications can either be incomplete or outdated [17]. Because of this, we manually established ground truth following a systematic three phase approach. (1) We began by collecting developer-provided specifications from the repositories. For the Django repositories, developer-provided specifications were unavailable. For them, we used DRF-Spectacular generated specifications as our starting point. (2) Two authors independently did a comprehensive manual code review to document all API endpoints, parameters, responses, and constraints. (3) Finally, the two authors reconciled their findings through detailed discussion to create a "ground truth" specification. We will discuss incomplete or outdated information in the developer-provided specifications in Section 4.2.

Our resulting specifications improve upon the ground truth from prior work [17] in the following ways:

- **Detailed Schemas:** While the previous ground truth often simplifies requests and responses by typing them as "object", our specification provides full schema definitions.

- **Spring Profile Support:** We correctly model the behavior of Spring profiles. For example, the CWA project contains external and ‘internal‘ profiles where the same method has different behaviors. The previous work missed that these profiles cannot be simultaneously active at runtime; we address this by creating two separate specifications.
- **Minor Corrections:** We rectified various smaller issues, such as parameter name inconsistencies. For instance, in the CatWatch project, we corrected a parameter from ‘sort_by‘ to ‘sortBy‘.

Evaluation Metrics. We adopt the evaluation metrics, proposed by Respector, and refine them further. We measure precision and recall on five dimensions:

(1) Endpoint methods. The fundamental unit of an API, evaluated as the correct identification of the `(path, HTTP_method)` tuple for each operation.

(2) Request parameters. For query and path parameters, we validate the `(name, location, data_type)` tuple as in Respector. But for body parameters, unlike Respector, we assess each individual `(field_name, data_type)` pair within its JSON schema. In Respector's evaluation, for example, a tool could be deemed perfect for identifying a parameter as a `User` object, even if the generated schema for the `User` object were missing fields like `name` or contained incorrect data types (*e.g.,* `name` is incorrectly marked as an `integer`).

(3) Response parameters. Similarly, we validate each `(field_name, data_type)` pair within the JSON body for each defined HTTP status code. Respector only validated the parameter type (*e.g.,* `User` object as discussed above).

(4, 5) Request and response constraints. Respector evaluates 8 OpenAPI constraints and that too on just query and path parameters. We evaluate a broader set of 13 common OpenAPI constraints: `type`, `default`, `required`, `enum`, `multipleOf`, `minLength`, `maxLength`, `minimum`, `maximum`, `minItems`, `maxItems`, `exclusiveMinimum`, and `exclusiveMaximum`,. We apply this evaluation to both: (a) constraints on query and path parameters and (b) constraints on individual fields within the JSON schemas of request body parameters and response bodies.

Experiment Procedure. For our main accuracy evaluation in RQ1 we use gpt-o4-mini, an economical reasoning model that provides strong reasoning capability with lower inference cost and higher throughput than other models. To answer RQ2, we conducted experiments using three reasoning LLMs (gpt-o1, gpt-o4-mini, DeepSeek-R1) and two non-reasoning LLMs (gpt-4.1, gpt-4.1-mini) to compare their performance. To account for the inherent stochasticity of LLMs and to ensure the robustness of our results, we performed three independent runs for each repository and LLM, and and reported the average results across these three runs. Alongside accuracy, our framework collects operational statistics, such as token usage, LLM costs, and end-to-end runtime for each run. We call each LLM using Microsoft Azure API, with a default sampling temperature of 0.5 (where applicable), a maximum output limit of 8000 tokens, and a retry limit of three to handle errors, such as transient errors, validation errors, and rate limiting errors.

4.2 Results and Analysis

4.2.1 RQ1: Can SPECULATE generate accurate specifications? Table 2 compares the precision and recall of SPECULATE, across the five dimensions, with DRF-spectacular for Django repositories, and with Respector and developer-provided specifications for Jersey and Spring Boot repositories. DRF-spectacular failed to generate specifications for Education backend and Libre Photos repositories. Developer-provided specification for enviroCar did not describe responses. All these metrics are

Table 2. Overall accuracy of Speculate vs. DRF-spectacular on Django repositories, and Respector and developer-provided specifications on Jersey and Spring-Boot repositories. GT: Ground Truth, P: Precision, R: Recall, N/A: Failed to Setup.

	Endpoint Method			Request Parameters			Request Constraints			Response Parameters			Response Constraints		
Tool	**GT**	**P**	**R**	**GT**	**P**	**R**	**GT**	**P**	**R**	**GT**	**P**	**R**	**GT**	**P**	**R**
Treeherder	**83**	**100.0**	**100.0**	**456**	**98.6**	**98.4**	**593**	**95.9**	**91.8**	**1218**	**83.7**	**69.9**	**1506**	**66.9**	**71.7**
DRF-Spectacular		97.6	97.6		61.9	40.6		49.2	39.3		71.5	50.3		40.0	49.3
Mathesar	**105**	**100.0**	**100.0**	**372**	**83.6**	**94.7**	**593**	**71.2**	**83.6**	**844**	**86.6**	**86.6**	**1562**	**84.3**	**79.9**
DRF-Spectacular		97.2	65.8		44.3	52.1		27.9	37.1		46.9	54.4		47.4	51.9
Education backend	**31**	**100.0**	**100.0**	**88**	**95.5**	**95.5**	**159**	**87.8**	**86.2**	**269**	**95.2**	**95.1**	**474**	**92.7**	**88.8**
DRF-Spectacular		N/A	N/A		N/A	N/A		N/A	N/A		N/A	N/A		N/A	N/A
Libre Photos	**163**	**100.0**	**100.0**	**498**	**91.9**	**89.9**	**846**	**83.0**	**76.0**	**2796**	**87.7**	**75.6**	**4383**	**66.1**	**62.7**
DRF-Spectacular		99.4	100.0		42.8	78.2		18.1	29.8		83.0	52.5		71.8	55.1
Average	**–**	**100.0**	**100.0**	**–**	**92.4**	**94.6**	**–**	**84.5**	**84.4**	**–**	**88.3**	**81.8**	**–**	**77.5**	**75.8**
DRF-Spectacular	–	98.1	87.8	–	49.7	57.0	–	31.7	35.4	–	67.1	52.4	–	53.1	52.1
Senzing	**34**	**100.0**	**100.0**	**156**	**100.0**	**99.8**	**155**	**72.4**	**98.1**	**1901**	**84.0**	**82.0**	**2081**	**58.4**	**82.4**
Respector		100.0	100.0		100.0	98.7		100.0	93.1		97.1	1.7		97.1	1.6
Dev Spec		100.0	100.0		95.6	98.7		92.1	98.1		99.7	98.8		99.9	91.8
enviroCar	**128**	**100.0**	**100.0**	**606**	**97.4**	**71.2**	**900**	**88.0**	**69.5**	**2262**	**67.5**	**42.3**	**3006**	**59.1**	**40.2**
Respector		100.0	100.0		100.0	32.0		99.7	40.7		86.3	3.6		86.3	2.7
Dev Spec		97.5	60.2		96.6	32.4		78.2	19.5		N/A	N/A		N/A	N/A
Digdag	**41**	**100.0**	**100.0**	**94**	**92.0**	**97.9**	**147**	**87.7**	**97.2**	**459**	**99.7**	**98.6**	**757**	**94.6**	**96.9**
Respector		100.0	100.0		100.0	85.1		100.0	77.6		84.1	8.0		84.1	4.9
Dev Spec		100.0	95.1		89.0	86.2		82.5	76.8		15.2	53.8		13.4	32.6
Gravitee	**28**	**100.0**	**100.0**	**400**	**100.0**	**100.0**	**592**	**82.0**	**94.8**	**1414**	**95.0**	**93.7**	**1881**	**81.1**	**90.1**
Respector		100.0	100.0		100.0	8.5		96.9	10.3		41.4	14.9		41.4	11.2
Dev Spec		92.8	92.8		28.2	20.3		27.9	22.5		47.8	19.0		43.8	16.3
Kafka	**69**	**100.0**	**100.0**	**204**	**83.0**	**86.9**	**377**	**84.7**	**88.3**	**751**	**90.6**	**56.5**	**1368**	**92.5**	**53.4**
Respector		100.0	100.0		99.4	75.0		95.2	72.7		78.6	1.5		78.6	0.8
Dev Spec		89.1	59.4		77.6	61.3		82.0	59.0		90.0	78.7		91.0	76.6
Cassandra	**50**	**100.0**	**100.0**	**115**	**99.7**	**91.9**	**194**	**98.9**	**87.8**	**84**	**90.0**	**71.8**	**86**	**82.0**	**70.9**
Respector		100.0	100.0		100.0	27.8		100.0	16.5		18.8	10.7		18.8	10.5
Dev Spec		100.0	100.0		100.0	86.9		100.0	82.5		88.6	46.4		87.0	46.5
Features Service	**18**	**100.0**	**100.0**	**35**	**97.8**	**100.0**	**58**	**92.6**	**100.0**	**31**	**100.0**	**91.4**	**35**	**92.0**	**83.6**
Respector		100.0	100.0		100.0	100.0		100.0	100.0		46.0	51.6		46.0	50.0
Dev Spec		100.0	100.0		100.0	100.0		100.0	100.0		100.0	88.6		96.9	56.4
RESTCountries	**27**	**100.0**	**100.0**	**37**	**100.0**	**100.0**	**71**	**100.0**	**88.3**	**1162**	**100.0**	**99.7**	**1163**	**100.0**	**94.7**
Respector		100.0	100.0		100.0	100.0		100.0	80.0		95.7	5.8		95.7	5.8
Dev Spec		100.0	37.0		100.0	54.1		100.0	42.3		0.0	0.0		0.0	0.0
Ur-Codebin	**7**	**100.0**	**85.7**	**16**	**100.0**	**87.5**	**71**	**100.0**	**90.1**	**30**	**100.0**	**100.0**	**36**	**100.0**	**100.0**
Respector		100.0	85.7		100.0	37.5		100.0	29.7		12.0	20.0		12.0	16.7
Dev Spec		100.0	100.0		100.0	100.0		87.9	78.4		70.0	70.0		43.8	58.3
Quartz-Manager	**11**	**100.0**	**91.0**	**19**	**100.0**	**79.0**	**38**	**94.1**	**84.2**	**74**	**100.0**	**91.9**	**90**	**100.0**	**93.3**
Respector		71.4	91.0		100.0	15.8		100.0	20.0		70.8	23.0		70.8	18.9
Dev Spec		100.0	100.0		100.0	26.3		100.0	26.3		0.0	0.0		0.0	0.0
Catwatch	**14**	**100.0**	**100.0**	**76**	**100.0**	**100.0**	**93**	**100.0**	**85.0**	**145**	**100.0**	**100.0**	**153**	**98.0**	**97.4**
Respector		100.0	100.0		100.0	42.1		100.0	43.8		100.0	15.9		100.0	15.9
Dev Spec		100.0	42.9		92.7	34.2		81.8	31.8		29.7	18.6		29.7	18.6
CWA Internal	**3**	**100.0**	**100.0**	**5**	**83.3**	**100.0**	**8**	**81.8**	**100.0**	**8**	**100.0**	**100.0**	**12**	**100.0**	**83.3**
Respector		60.0	100.0		25.0	20.0		20.0	12.5		40.0	25.0		40.0	16.7
Dev Spec		60.0	100.0		35.7	100.0		38.1	100.0		53.9	87.5		41.2	58.3
CWA External	**3**	**100.0**	**100.0**	**10**	**100.0**	**100.0**	**16**	**100**	**100.0**	**11**	**100.0**	**100.0**	**17**	**100.0**	**100.0**
Respector		60.0	100.0		50.0	20.0		40.0	13.3		20.0	9.1		20.0	5.9
Dev Spec		60.0	100.0		71.4	100.0		71.4	100.0		84.6	100.0		88.2	100.0
ProxyPrint	**68**	**100.0**	**100.0**	**182**	**99.4**	**90.1**	**253**	**81.8**	**87.0**	**663**	**93.6**	**79.9**	**668**	**66.6**	**80.0**
Respector		90.7	100.0		78.1	27.5		85.5	37.2		25.8	3.5		25.8	3.5
Dev Spec		91.6	95.6		96.0	65.4		96.4	63.8		25.6	6.0		25.5	6.0
Ohsome API	**159**	**100.0**	**100.0**	**1942**	**93.3**	**78.0**	**2129**	**88.2**	**67.3**	**2875**	**90.4**	**69.8**	**2872**	**53.0**	**67.7**
Respector		100.0	100.0		47.5	46.8		39.2	28.6		91.1	17.2		91.1	17.2
Dev Spec		100.0	84.9		47.0	39.2		39.0	25.0		72.2	70.5		48.3	71.9
OCVN	**278**	**100.0**	**96.4**	**5062**	**92.8**	**85.7**	**5824**	**92.8**	**85.0**	**12056**	**95.2**	**72.1**	**12777**	**86.2**	**70.7**
Respector		97.5	99.3		51.7	8.6		17.2	2.5		67.8	5.9		67.8	6.4
Dev Spec		97.9	68.9		48.9	28.5		49.2	22.7		32.9	37.4		32.8	36.4
Average	**–**	**100.0**	**98.3**	**–**	**96.2**	**91.8**	**–**	**90.3**	**88.9**	**–**	**94.1**	**84.4**	**–**	**85.2**	**81.5**
Respector	–	92.5	98.5	–	84.5	46.6	–	80.9	42.4	–	61.0	13.6	–	61.0	11.8
Dev Spec	–	93.1	83.5	–	79.9	64.6	–	76.7	59.3	–	54.0	51.7	–	49.4	44.6

shown as N/A in Table 2. We first summarize overall findings for each dimension and then present repository-wise case studies.

Both Respector and Speculate use identical methods to identify endpoint methods. Both of them achieve 100% precision and recall for endpoint methods on Jersey and Spring Boot repositories.

SPECULATE provides 100% precision and recall for endpoint methods on Django repositories as well, but DRF-spectacular has a lower precision (−2 %) and recall (−12 %). This is because, as discussed in Section 3, SPECULATE starts each server and uses Django's `EndpointEnumerator` class to identify all registered URL patterns (endpoints). Whereas, DRF-spectacular uses a completely static-analysis-based approach to look for `ViewSet` and `ApiView` subclasses, and therefore misses dynamically generated endpoints.

For the two Django repositories, SPECULATE has much better precision and recall than DRF-spectacular across all the dimensions.

For Jersey and Spring Boot repositories, SPECULATE usually has better precision and significantly better recall than Respector for all the repositories across all the dimensions. For responses, SPECULATE has significantly better recall. On average, the difference in precision and recall, shown as (p%, r%), are as follows: request parameters (+12%, +45%), request constraints (+9%, +47%), response parameters (+33%, +71%), and response constraints (+24%, +70%).

For Django repositories, DRF-spectacular incorrectly generates specifications whenever the developer uses custom logic, *i.e.,* deviates from Django templates as discussed in Section 2.2. SPECULATE makes similar errors for Django, Jersey and Spring Boot repositories. Therefore, we focus the rest of our disccusion on Jersey and Spring Boot repositories, and focus more on errors by SPECULATE, since errors by Respector were discussed in Sections 2.3 and 2.4.

Incomplete Schema for Complex Objects. The most frequent error category involves the LLM's failure to completely specify deeply nested JSON schemas, often marking them as a generic object. For example, in enviroCar the `POST /tracks` endpoint takes a complex JSON object `TrackCreate` as a body parameter with multiple levels of nesting. `TrackCreate` has three fields: `type`, `properties`, and `features`. The field `TrackCreate > properties` itself has 8 nested fields. The field `TrackCreate > features` is of type `MeasurementCreate` which again has three nested fields: `type`, `geometry`, and `properties`. The field `TrackCreate > features > properties` is again a complex object with three fields `time`, `sensor`, and `phenomenons`. Similarly, `TrackCreate > features > geometry` and `TrackCreate > features > properties > phenomenons` are also complex objects. Respector marks the request parameter of `POST /tracks` as simply an `object`, ignoring its deeply nested structure. SPECULATE is able to correctly identify the first-level of the object, but marks deeper levels like `TrackCreate > features` as simply an `object`. This pattern of failing on deeply nested schemas is the primary reason for the low recall scores in this repository. The impact is particularly evident for response parameters and constraints, where Respector's recall is very low at 3.6% and 2.7% respectively. While SPECULATE performs significantly better with 42.3% and 40.2% recall, its scores are still limited because these components are often reused, meaning a single missing field or incorrect type negatively affects many endpoints. Similar issues occurred in the Senzing repository, where low recall was also due to the LLM marking a heavily reused, deeply nested component (*e.g.,* `SzEntityData`) as an object.

Insufficient Code Context. Another class of errors stems from SPECULATE failing to provide sufficient context to the LLM. For the `POST /resetPassword` endpoint in enviroCar, SPECULATE failed to send all the relevant code context. In particular, SPECULATE traverses the call graph till a depth of 3 calls, and a critical class `UserDecoder` was not sent. Because of this, the LLM guessed that a `user` field was of a `User` type, but it was actually just a `string`.

Dynamic Behavior Beyond Code Analysis. SPECULATE is limited when behavior is defined dynamically outside the source code. For the REST countries repository, both Respector and SPECULATE fail to identify that the `region` path parameter is an `enum` with values `Africa`, `Americas`, etc. This

happens because the code dynamically loads regions from a `countriesV2.json` file. Both Respector and Speculate do not analyze JSON and other configuration files.

Naming and Reference Errors. We also observed errors related to incorrect naming and broken references. For the `GET /measurements` endpoint in enviroCar, the LLM responds with a reference to a `Measurements` component, instead of the correct `Measurement` component (notice the extra `s` in the end). This fails Speculate's referential integrity validation. Even after retries, the LLM could not fix this mistake. A validation fix based on lexicographic similarity would have fixed this issue. Similarly, in the Kafka repository, for the response of `/topics/topic/partitions/partition: POST`, the `key_schema_id` field was incorrectly named `keySchemaId`

Incorrect Content-Type Deduction. In other cases, the LLM fails to correctly resolve information that is present in the context. For the `/consumers/group:post` endpoint in the Kafka repository, the LLM incorrectly deduced the Content-Type as `application/json` instead of the required vendor-specific media type, `application/vnd.kafka.v2+json`. This error occurred even though the context provided the `Versions.java` file defining the correct media type; the LLM failed to perform the cross-file constant resolution and defaulted to the more common type.

Misinterpretation of In-Code Artifacts. Occasionally, the LLM is misled by in-code artifacts that are not part of the API contract. For example, in the response of Cassandra's `/api/v0/metadata/endpoints:get`, the LLM's reasoning was heavily influenced by the following string constant found directly within the `MetadataResources.java` source code:

```
private static final String ENDPOINTS_RESPONSE_EXAMPLE = "{\n" +
"␣"entity":␣[\n" +
"␣{\n" +
"␣"DC":␣"datacenter1",\n" +
//... (rest of the example)
```

While technically correct for an internal server object, the example is contextually misleading for an API contract. It shows the server-side JAX-RS Response object used for debugging, not the actual JSON payload sent to the client. Biased by this example, the LLM incorrectly modeled the entire debug structure instead of the true payload.

Incorrect Nesting Level. Certain errors arise from the LLM generating an incorrect schema structure. For instance, in the Kafka repository's /v3/clusters:GET response, the LLM generated a schema with a superfluous wrapper object around the actual data structure. In the source code, the @JsonValue annotation from the Jackson library directs the serializer to flatten this object. However, the generated schema did not reflect this flattening behavior, resulting in an incorrect nesting level.

Our analysis also revealed gaps in developer-provided specifications. In enviroCar, the specification omits many nested endpoints identified by Speculate, which are generated using JAX-RS Sub-Resource Locator patterns. Developers documented primary endpoints like `/tracks/id` but missed derived paths such as `/users/id/tracks/id/measurements/id`. The specification for REST Countries omitted response schemas and the v1 API endpoints entirely. In Features Service, the developer-provided specification omitted the `required: True` constraint on many schema fields (for example, the `name` field of the `Features` component), which were detected by Speculate, making the specification more rich and accurate.

The preceding analysis identified several recurring error categories, such as incomplete schema generation and misinterpretation of in-code artifacts. To quantify the distribution of these failure modes, we conducted a systematic error analysis. Manually identifying and classifying every property affected by an error across our entire dataset is a labor-intensive task. Therefore, we

Table 3. Taxonomy of errors across 5 repositories.

Error Type	Count of Properties affected
Kafka	
Incorrect Nesting Level	3181
Incorrect naming of the fields	158
Lack of Schema Detail	157
Incorrect content type deduction	78
Incorrect response code	10
Subtotal	*3584*
Cassandra	
Lack of Schema Detail	142
Misinterpretation of non-contextual evidence	138
Incorrectly predicted properties	14
Incorrect response type	4
Subtotal	*298*
Catwatch	
Lack of Schema Detail	11
Incorrect content type deduction	7
Subtotal	*18*
Education Backend	
Insufficient Code Context	39
Lack of Schema Detail	31
Incorrect content type deduction	3
Subtotal	*73*
Proxy Print	
Lack of Schema Detail	28
Incorrect content type deduction	13
Insufficient Code Context	3
Subtotal	*44*
Total Errors	**4017**
Total Runs	**15**

performed this meticulous classification on five repositories (one Django, two Jersey, and two Spring Boot), randomly chosen to reflect the diversity of frameworks and languages in our study.

Table 3 presents this taxonomy, breaking down the errors by type for each repository. Our analysis reveals that the impact of errors is not evenly distributed. A single error category in one repository—'Incorrect Nesting Level' in Kafka—accounts for 3,181 affected properties, comprising over 81% of all errors documented in the subset.

Findings to RQ1: *The F1 score of SPECULATE is 0.87, which is better than existing static-analysis-based tools (Respector: 0.50, DRF-spectacular: 0.57). This is because SPECULATE always has a better recall but its precision can be slightly worse than a sophisticated static-analysis-based tool. While static analysis tools make mistakes due to non-template code, dynamic control flow, knowledge boundaries, and path explosion, as detailed in Section 2, SPECULATE makes mistakes due to either sending incomplete code in prompts or LLMs just failing to generate specifications correctly. We also find various mistakes in developer-provided specifications.*

RQ2: How does the choice of LLMs especially reasoning and non-reasoning models impact SPECULATE's accuracy? We find that reasoning models provide only slightly higher detection rates than non-reasoning models, but with significantly higher cost. Figure 2 shows minimal variance across configurations, demonstrated by the small percentage increase from the lowest to the highest detection rate for each: request parameters (7.56%), request constraints (3.49%), response parameters (3.03%), and response constraints (2.91%). This indicates all LLM configurations detect most API properties successfully.

However, reasoning models excel at handling imperative code patterns. For the `/api/project/{project}/jobs/{id}/similar_jobs/` endpoint in Treeherder (Listing 4), which imperatively generates over 40 request parameters through a `FilterClass`, manually renames parameters, and transforms inputs without using Django REST Framework's declarative features. `gpt-o1` consistently identifies all parameters across three runs, while `Deepseek-R1` misses only the renamed parameters. `gpt-o4-mini` shows high variance, sometimes missing dynamic filters, and both non-reasoning models (`GPT-4.1 Mini`, `GPT-4.1`) fail to detect the `JobFilter` parameters entirely.

The cost-benefit tradeoff is substantial: `gpt-o1` costs about US$100 per repository versus US$1 for `GPT-4.1 Mini` (100× increase), while the average execution time per repository ranges from 13 minutes (`GPT-4.1 Mini`) to 80 minutes (`Deepseek-R1`). For declaratively written APIs, non-reasoning models suffice. For complex, imperative codebases, reasoning models provide the essential accuracy improvements.

```python
def list(self, request, project):
    # manual renaming of the incoming parameter names
    for param_key, param_value in request.query_params.items():
        # replace `result_set_id` with `push_id`
        if param_key.startswith("result_set_id"):
            new_param_key = param_key.replace("result_set_id", "push_id")
            filter_params[new_param_key] = param_value
        # ...

    #imperatively checking for parameters
    try:
        offset = int(filter_params.get("offset", 0))
        count = int(filter_params.get("count", 10))
    except ValueError:
        return Response("Invalid value for offset or count", status=HTTP_400_BAD_REQUEST)

    # ...
    #defines a large set of explicit filters
    jobs = JobFilter(
        {k: v for (k, v) in filter_params.items()},
        queryset=Job.objects.filter(repository=repository).select_related(
            *self._default_select_related
        ),
    ).qs
    # ...


class JobFilter(django_filters.FilterSet):
    # filters dynamiclly generated via the Meta class, such as 'in' lookups.
    id = django_filters.NumberFilter(field_name="id")
    id__in = NumberInFilter(field_name="id", lookup_expr="in")
    #more than 25 fields
    # ...
    class Meta:
        model = Job
        # this allows for automatically generate filters for the model's fields
        fields = {
            "last_modified": ["lt", "lte", "exact", "gt", "gte"],
            "submit_time": ["lt", "lte", "exact", "gt", "gte"],
            #...
        }
        #...
```

Listing 4. Implementation of the `GET /api/project/{project}/jobs/{id}/similar_jobs/` endpoint in Treeherder.

Findings to RQ2: *Reasoning models provide only slightly better F1 score than non-reasoning models, but with the dollar cost increased by a factor of 102 times (US$1.02 → US$104 and end-to-end runtimes increased by a factor of 5.9 (13m 32s → 80m 13s).*

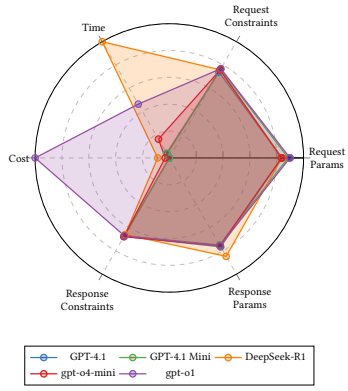
4.3 Threats to Validity

We mitigate the threat to internal validity by multiple authors independently developing ideal specifications from code and then reconciling their analysis.

To mitigate threats to external validity, we used the Jersey and Spring Boot repositories from a previous work [17], and we selected Django repositories that are both popular, which is measured by GitHub stars, and actively maintained, which is based on recent commits. We believe these selection criteria minimize potential bias in our results.

The accuracy of Speculate is largely determined by the accuracy of OpenAPI specifications generated by the LLMs. We studied the impact of using different LLMs on the accuracy and found them to be largely similar, with reasoning models slightly outperforming non-reasoning models. LLMs may provide perfect specifications if they had already seen the specifications in their

Fig. 2. LLM performance comparison across models and configurations.



training datasets. For 4 out of the 19 repositories in our dataset, there were no developer-provided specifications. Further, our results show that even for the 15 remaining repositories, LLMs generated richer and more accurate specifications than the ones provided by the developers.

Since LLMs are non-deterministic in nature, we repeated each experiment 3 times and reported the average accuracy. LLM versions might change over time, thereby providing different accuracy with different runtimes and costs, making long-term reproducibility of our results difficult. To verify our findings, we provide the source code of SPECULATE along with a complete trace of every conversation that SPECULATE had with LLMs for each repository. For each conversation, we also provide response times, along with estimated token counts and dollar costs.

5 DISCUSSION

While the LLM-based tool SPECULATE provides better accuracy than a sophisticated static analysis tool Respector on real-world repositories, there exists room to further improve both precision and recall. Broadly, SPECULATE made two types of errors: (i) lack of LLM reasoning capabilities, and (ii) sending incomplete code context.

For (i), as LLMs improve in their reasoning capabilities, the accuracy of SPECULATE might naturally improve without any additional development effort. In the interim, it could also be interesting to try to use static analysis to automatically fix invalid LLM outputs. SPECULATE currently fixes only simple validation failures.

For (ii), it will be interesting future work to attempt to offload SPECULATE's orchestration–processing components in a topological order and then processing endpoints using hard-coded prompts– to LLMs as well. We might expose the code index using model context protocol to an LLM agent. It might be interesting to evaluate the cost and if this approach can provide a better accuracy. In particular, does it automatically solve current limitations of SPECULATE, such as not sending configuration files?

6 RELATED WORK

SPECULATE belongs to the rapidly growing family of LLM-based software engineering tools such as KNighter [37] for synthesizing static analysis checkers, LLift [20] for identifying use-before-initialization bugs, [8] for inferring error specifications, Vercation [9] for identifying vulnerable software versions, and AdvSCanner [35] for generating adversarial smart contracts to exploit vulnerabilities.

Speculate automatically generates OpenAPI specifications from code. This is complementary to other related tasks like designing, building, modeling, testing, and validation of API implementations [2, 4, 22, 24, 28].

Techniques outside of Speculate and our static-analysis tools (used as baselines in this paper) fall into two broad streams.

(i) Developer-driven capture. Tools such as Swagger Inspector, Swagger Core, and their relatives—SpringFox, BlueBird, ramlo, Talend, springdoc-openapi, and others—rely on some form of *manual* effort. Developers either annotate the code or explicitly invoke each endpoint while the recorder logs request– response pairs, yielding an OpenAPI "skeleton" that must subsequently be enriched by hand [5, 23, 27, 29, 30, 32, 38]. The generated documents typically omit alternative responses, authentication details, and fine-grained parameter constraints.

(ii) Black-box inference. Other systems treat the server as opaque and learn only from runtime traffic. ApiCarv, AppMap, and ExpressO intercept the HTTP exchanges produced by an existing UI- or API-test suite; ApiCarv additionally sends lightweight probes to mutate paths and verbs for extra coverage [3, 25, 36]. Because such approaches can describe only what the trace actually exercises, they achieve high precision yet can miss unexecuted endpoints or rare parameter combinations, leading to moderate recall. Moreover, without source visibility they cannot determine whether an observed field is mandatory, what ranges or formats are enforced, or which hidden parameters exist, so the inferred specifications remain largely skeletal.

While Respector [17] is unique in finding OpenAPI specifications from code for Java-based frameworks, there are many framework-specific tools similar to DRF-spectacular [14] for Django. For example, springdoc-openapi [27] generates specifications for the Java-based SpringBoot framework, ExpressO [25] for Javascript-based Express framework and rswag [11] for Ruby-on-Rails. In this paper, we have demonstrated that Speculate can generalize easily across frameworks and languages; only a framework-specific code indexer needed to be built to use Speculate over Java-based Jersey, Spring Boot and Python-based Django repositories.

7 CONCLUSION

We presented Speculate for automatically generating OpenAPI specifications from their implementations. Unlike existing tools, Speculate heavily relies on LLMs making it more easily generalize to languages and frameworks. Our evaluations show that while Speculate's precision can be a bit lower than state-of-the-art tools, its recall is usually much higher. Maintaining Speculate could be more practical than sophisticated static-analysis-based tools, as it does not require deep expertise, such as in SMT solvers. Speculate's accuracy might improve naturally as LLMs get better at reasoning tasks.

DATA AVAILABILITY

All source code and experimental data for our tool is publicly available at [1].

REFERENCES

[1] 2026. Speculate. https://doi.org/10.5281/zenodo.19185245.
[2] Apiary. 2024. Dredd API Testing Framework. https://dredd.org/en/latest/. Accessed: July 15, 2025.
[3] AppMap. 2022. AppMap. https://appmap.io/. Online; accessed July-2025.
[4] Andrea Arcuri. 2019. RESTful API Automated Test Case Generation with EvoMaster. *ACM Trans. Softw. Eng. Methodol.* 28, 1, Article 3 (Jan. 2019), 37 pages. https://doi.org/10.1145/3293455
[5] Djordje Atlialp and Mathias Polligkeit. 2019. BlueBird. https://github.com/KittyHeaven/blue_bird. Online; accessed July-2025.
[6] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. RestTestGen: Automated Black-Box Testing of RESTful APIs. In *Proceedings of the 41st International Conference on Software Engineering (ICSE).* IEEE/ACM, 1039–1050.

https://doi.org/10.1109/ICSE.2019.00109

[7] Justus Bogner, Johannes Fritzsch, Stefan Wagner, and Alfred Zimmermann. 2016. A Survey on the Current State of REST API Documentation. In *Proceedings of the 2nd International Workshop on API Usage and Evolution (API-EVOL)*. IEEE, 13–19. https://doi.org/10.1109/API-EVOL.2016.5

[8] Patrick J. Chapman, Cindy Rubio-González, and Aditya V. Thakur. 2024. Interleaving Static Analysis and LLM Prompting. In *Proceedings of the 13th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis (SOAP 2024)*. https://doi.org/10.1145/3652588.3663317

[9] Yiran Cheng, Lwin Khin Shar, Ting Zhang, Shouguo Yang, Chaopeng Dong, David Lo, Shichao Lv, Zhiqiang Shi, and Limin Sun. 2024. LLM-Enhanced Static Analysis for Precise Identification of Vulnerable OSS Versions. arXiv:2408.07321 [cs.SE]

[10] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008) (Lecture Notes in Computer Science, Vol. 4963)*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer, Berlin, Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

[11] DomainDrivenDev. 2018. Rswag. https://github.com/rswag/rswag.

[12] Eclipse Foundation. 2025. HK2: JSR-330 Dependency Injection for Java. https://javaee.github.io/hk2/.

[13] Hamza Ed-douibi, Javier Luis Cánovas Izquierdo, and Jordi Cabot. 2018. Automatic Generation of Test Cases for REST APIs: a Specification-Based Approach. In *2018 IEEE 22nd International Enterprise Distributed Object Computing Conference (EDOC)*. IEEE, 181–190.

[14] Tino Franzel. 2020. drf-spectacular. https://github.com/tfranzel/drf-spectacular.

[15] Julio Vilmar Gesser, Federico Tomassetti, and Romain Strokes. 2018. JavaParser: A Lexer and Parser for Java 10. In *Proceedings of the 12th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '18)*. ACM, 91–93. https://doi.org/10.1145/3239372.3239389

[16] Google LLC. 2025. Guice. https://github.com/google/guice. Online; accessed July-2025.

[17] Ruikai Huang, Manish Motwani, Idel Martinez, and Alessandro Orso. 2024. Generating REST API Specifications through Static Analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 107, 13 pages. https://doi.org/10.1145/3597503.3639137

[18] Stefan Karlsson, Adnan Causevic, and Daniel Sundmark. 2020. QuickREST: Property-based Test Generation of OpenAPI-Described RESTful APIs . In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE Computer Society, Los Alamitos, CA, USA, 131–141. https://doi.org/10.1109/ICST46399.2020.00023

[19] Alexander Lercher, Christian Macho, Clemens Bauer, and Martin Pinzger. 2024. Generating Accurate OpenAPI Descriptions from Java Source Code. arXiv:2410.23873 [cs.SE] https://arxiv.org/abs/2410.23873

[20] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 111 (2024). https://doi.org/10.1145/3649828

[21] Mozilla. [n. d.]. Treeherder (specific commit). https://github.com/mozilla/treeherder/tree/542c0e83843a8c403abffe072c8dc94b0dc943a3. Accessed July 2025.

[22] Oracle Corporation. 2024. Apiary API Design & Documentation. https://apiary.io/. Accessed: July 15, 2025.

[23] Marty Pitt, Dilip Krishnan, and Adrian Kelly. 2020. SpringFox. https://github.com/springfox/springfox. Online; accessed July-2025.

[24] Postman, Inc. 2024. Postman API Platform. https://www.postman.com/. Accessed: July 15, 2025.

[25] Alessandro Romanelli, Souhaila Serbout, and Cesare Pautasso. 2022. ExpressO: From Express.js Implementation Code to OpenAPI Interface Descriptions. In *Proceedings of the European Conference on Software Architecture (ECSA)*. Springer.

[26] Senzing community. 2022. Senzing REST API. https://github.com/Senzing/senzing-api-server. Online; accessed March 2023.

[27] springdoc. 2023. springdoc-openapi. https://github.com/springdoc/springdoc-openapi. Online; accessed July-2025.

[28] Stoplight. 2024. Stoplight API Design and Documentation Platform. https://stoplight.io/. Accessed: July 15, 2025.

[29] Swagger. 2023. Swagger Core. https://github.com/swagger-api/swagger-core. Online; accessed July-2025.

[30] Swagger. 2023. Swagger Inspector. https://inspector.swagger.io/. Online; accessed July-2025.

[31] Swagger. 2025. Swagger Codegen. https://github.com/swagger-api/swagger-codegen. Online; accessed July-2025.

[32] Talend S.A. 2023. Talend. https://www.talend.com. Online; accessed July-2025.

[33] The enviroCar project. 2022. enviroCar Server. https://github.com/enviroCar/enviroCar-server. Online; accessed March-2023.

[34] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundaresan. 2010. Soot: A Java Bytecode Optimization Framework. In *Proceedings of the 2010 Conference of the Center for Advanced Studies on Collaborative Research (CASCON '10)*. ACM, Toronto, Ontario, Canada, 214–224. https://doi.org/10.1145/1925805.1925818

[35] Yin Wu, Xiaofei Xie, Chenyang Peng, Dijun Liu, Hao Wu, Ming Fan, Ting Liu, and Haijun Wang. 2024. AdvSCanner: Generating Adversarial Smart Contracts to Exploit Reentrancy Vulnerabilities Using LLM and Static Analysis. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. 1019–1031. https://doi.org/10.1145/3691620.3695482

[36] Rahulkrishna Yandrapally, Saurabh Sinha, Rachel Tzoref-Brill, and Ali Mesbah. 2023. Carving UI Tests to Generate API Tests and API Specification. arXiv:2305.14692 [cs.SE] https://arxiv.org/abs/2305.14692

[37] Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. 2025. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers. arXiv:2503.09002 [cs.SE]

[38] Kamil Zasada and Michał Myśliwiec. 2016. ramlo. https://github.com/PGSSoft/ramlo. Online; accessed July-2025.